

COMP90038
Algorithms and Complexity

Cezary Kaliszyk
(thanks to Harald Søndergaard)

Lecture 16

Time/Space Tradeoffs—String Search Revisited

Announcements

September 25 is a holiday

Replacement lecture on Wednesday, Sep 23
PAR-192-L2-L108-Laby Theatre, 2pm

September 28 Week - no teaching

no lectures, tutorials, consultation hours and student hub

My consultation hours

Thursdays 10am, both in person and on Zoom, link in LMS

Last Lecture

Balanced trees

- AVL trees
- 2–3 trees

Spending Space to Save Time

Decreasing the time required to solve a problem

using additional memory in a clever way

Spending Space to Save Time

Decreasing the time required to solve a problem

using additional memory in a clever way

Naive way to compute the nth Fibonacci number

exponential (Lecture 6)

Spending Space to Save Time

Decreasing the time required to solve a problem

using additional memory in a clever way

Naive way to compute the nth Fibonacci number

exponential (Lecture 6)

Use memory

- tabulate the function FIB as we go
- When any intermediate FIB(i) is found, place it in a table
- Each call first looks in this table, compute only if not yet there

Fibonacci Numbers with Tabulation

Assume table F initialized with zeros

function FIB(n)
    if $n = 0$ **or** $n = 1$ **then**
        return 1
    $result \leftarrow F[n]$
    if $result = 0$ **then**
        $result \leftarrow \text{FIB}(n-1) + \text{FIB}(n-2)$
        $F[n] \leftarrow result$
    return $result$

(Same complexity as Lecture 6, but shows the principle)

Sorting by Counting

Large array, but small, fixed set of keys

Lots of duplicate keys, $k \ll n$

Sorting by Counting

Large array, but small, fixed set of keys

Lots of duplicate keys, $k \ll n$

For example, suppose all keys are single digits:

6 3 3 8 1 0 8 7 9 2 5 3 5 3 1 8 7 6 5 1 2 1 5 3

Sorting by Counting

Large array, but small, fixed set of keys

Lots of duplicate keys, $k \ll n$

For example, suppose all keys are single digits:

6 3 3 8 1 0 8 7 9 2 5 3 5 3 1 8 7 6 5 1 2 1 5 3

key	0	1	2	3	4	5	6	7	8	9
Occ	1	4	2	5	0	4	2	2	3	1

Sorting by Counting (why cumulative?)

We can now create a sorted array $S[1]..S[n]$ of the items by simply slotting items into pre-determined slots in S (a third linear scan).

6 3 3 8 1 0 8 7 9 2 5 3 5 3 1 8 7 6 5 1 2 1 5 3

key	0	1	2	3	4	5	6	7	8	9
Occ	1	4	2	5	0	4	2	2	3	1

Sorting by Counting (why cumulative?)

We can now create a sorted array $S[1]..S[n]$ of the items by simply slotting items into pre-determined slots in S (a third linear scan).

6 3 3 8 1 0 8 7 9 2 5 3 5 3 1 8 7 6 5 1 2 1 5 3

key	0	1	2	3	4	5	6	7	8	9
Occ	1	4	2	5	0	4	2	2	3	1

key	0	1	2	3	4	5	6	7	8	9
Occ (accum.)	1	5	7	12	12	16	18	20	23	24

Sorting by Counting (Code)

key	0	1	2	3	4	5	6	7	8	9
Occ (accum.)	1	5	7	12	12	16	18	20	23	24

for $i \leftarrow n$ **to** 1 **do**

$S[Occ[A[i]]] \leftarrow A[i]$

$Occ[A[i]] \leftarrow Occ[A[i]] - 1$

Sorting by Counting

- linear-time sorting algorithm
  - (for the cost of some extra space)
- works when we have a small range of keys
  - keys known in advance
- never performs key-to-key comparisons
- time complexity of key-comparison based sorting proven to be in $\Omega(n \log n)$.

String Matching Revisited

String Matching Applications

text editors, natural language processors, virus scanning, websearch engines

In Lecture 5

brute-force approach

String Matching Revisited

String Matching Applications

text editors, natural language processors, virus scanning, websearch engines

In Lecture 5

brute-force approach

Better?

- Observation: usually small, pre-determined alphabet
- We can pre-process the strings before the matching process starts
- Typically construct a small table
- Levitin refers to this as “input enhancement”

Horspool's String Search Algorithm

Comparing from right to left in the pattern.

Very good for random text strings.

S T R I . . . . . .
E X A M

Horspool's String Search Algorithm

Comparing from right to left in the pattern.

Very good for random text strings.

```
S T R I . . . . . .
E X A M
```

We can do better than just observing a mismatch here.

Because the pattern has no occurrence of I, we might as well slide it 4 positions along.

This decision is based only on knowing the pattern.

Horspool's String Search Algorithm

S T R I N G S E A R C H E X A M P
E X A M
E X A M

Horspool's String Search Algorithm

S T R I N G S E A R C H E X A M P
E X A M
E X A M
E X A M

Horspool's String Search Algorithm

S T R I N G S E A R C H E X A M P
E X A M
E X A M
E X A M
E X A M

Horspool's String Search Algorithm

S T R I N G S E A R C H E X A M P
E X A M
E X A M
E X A M
E X A M
E X A M

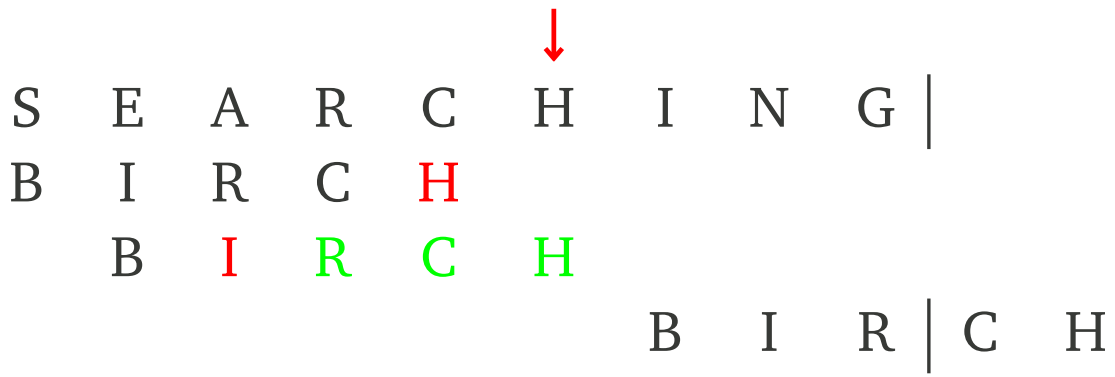
Horspool's String Search Algorithm

What happens when we have longer partial matches?

```
S E A R C H I N G |
B I R C H
  B I R C H
            B I R | C H
```

Horspool's String Search Algorithm

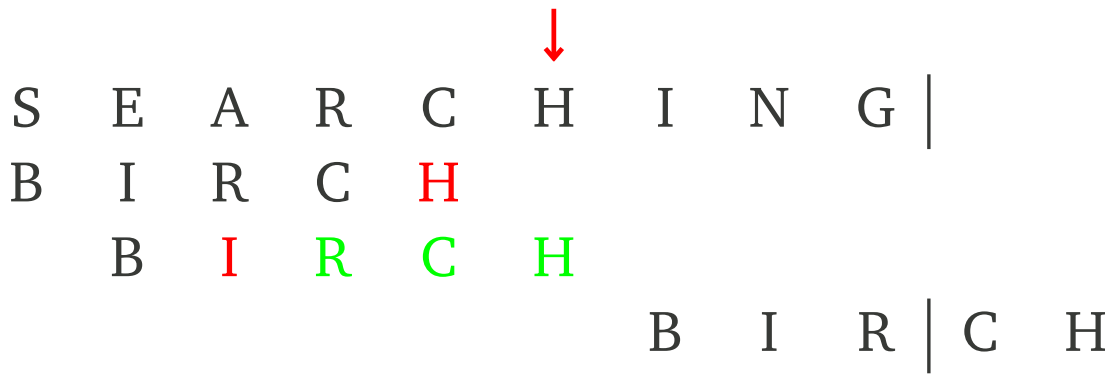
What happens when we have longer partial matches?



The shift is determined by the last character in the pattern.

Horspool's String Search Algorithm

What happens when we have longer partial matches?

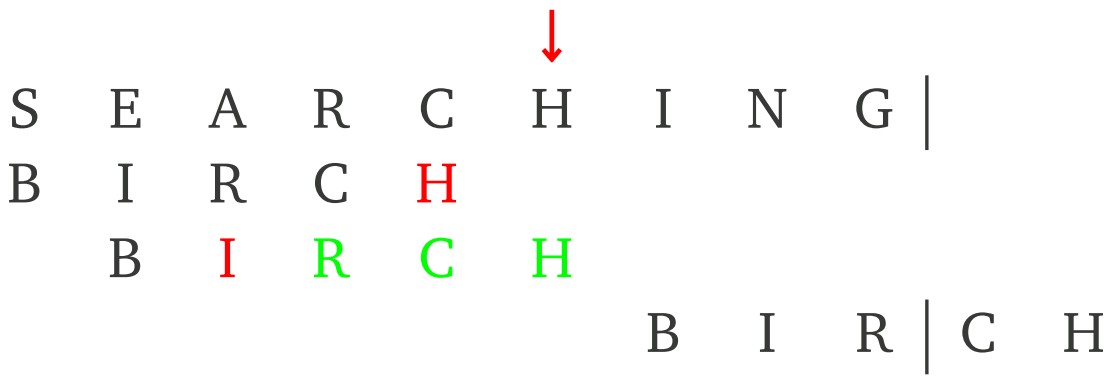


The shift is determined by the last character in the pattern.

Note that this is the same as the character in the text that we first matched against. Hence the skip is always determined by that character, whether it matched or not.

Horspool's String Search Algorithm

What happens when we have longer partial matches?



Char	Shift
A	5
B	4
C	1
⋮	⋮
H	5
I	3
⋮	⋮
R	2
S	5
⋮	⋮
Z	5

The shift is determined by the last character in the pattern.

Note that this is the same as the character in the text that we first matched against. Hence the skip is always determined by that character, whether it matched or not.

Horspool's String Search Algorithm

Building (calculating) the shift table is easy.

Horspool's String Search Algorithm

Building (calculating) the shift table is easy.

We assume indices start from 0.

Let *alphasize* be the size of the alphabet.

function FINDSHIFTS($P[\cdot], m$) ▷ Pattern P has length m
  for $i \leftarrow 0$ to $alphasize - 1$ **do**
    $Shift[i] \leftarrow m$

Horspool's String Search Algorithm

Building (calculating) the shift table is easy.

We assume indices start from 0.

Let *alphasize* be the size of the alphabet.

```
function FindShifts(P[·], m)          ▷ Pattern P has length m
    for i ← 0 to alphasize − 1 do
        Shift[i] ← m
    for j ← 0 to m − 2 do
        Shift[P[j]] ← m − (j + 1)
```

Horspool's String Search Algorithm

function HORSPOOL($P[\cdot], m, T[\cdot], n$)
    FINDSHIFTS(P, m)
    $i \leftarrow m-1$
    while $i < n$ **do**
        $k \leftarrow 0$
        while $k < m$ **and** $P[m-1-k] = T[i-k]$ **do**
            $k \leftarrow k+1$
        if $k = m$ **then** ▷ We have a match
            return $i-m+1$ ▷ Start of the match
        else
            $i \leftarrow i + Shift[T[i]]$ ▷ Slide the pattern along
    return -1

Horspool's String Search Algorithm

function HORSPOOL($P[\cdot], m, T[\cdot], n$)
    FINDSHIFTS(P, m)
    $i \leftarrow m-1$
    while $i < n$ **do**
        $k \leftarrow 0$
        while $k < m$ **and** $P[m-1-k] = T[i-k]$ **do**
            $k \leftarrow k+1$
        if $k = m$ **then** ▷ We have a match
            return $i-m+1$ ▷ Start of the match
        else
            $i \leftarrow i + Shift[T[i]]$ ▷ Slide the pattern along
    return -1

Optionally: post a sentinel

Append the pattern P to the end of the text T. Match guaranteed.

Horspool's String Search: Complexity



Horspool's String Search: Complexity



Worst case

still $O(m \times n)$ (like brute-force approach)

In practice (English texts)

Often fast, close to linear-time

Other Important String Search Algorithms

Boyer-Moore algorithm (BM)

- very similar
- more sophisticated shifting strategy
- $O(m+n)$

Knuth-Morris-Pratt algorithm (KMP)

- very good when the alphabet is small

Rabin-Karp algorithm (RK)

Soon

Knuth-Morris-Pratt (Not Examinable)

Suppose the alphabet is small (e.g., binary digits or nucleobases)

Knuth-Morris-Pratt (Not Examinable)

Suppose the alphabet is small (e.g., binary digits or nucleobases)

```
Text:     1 0 1 0 1 0 1 0 0
Pattern:  1 0 1 0 0
```

Every “false start” contains a lot of information.

Knuth-Morris-Pratt (Not Examinable)

Suppose the alphabet is small (e.g., binary digits or nucleobases)

```
Text:     1 0 1 0 1 0 1 0 0
Pattern:  1 0 1 0 0
```

Every “false start” contains a lot of information.

Can we pre-process the pattern to find out when the index i can be incremented by more than 1?

Knuth-Morris-Pratt (Not Examinable)

Suppose the alphabet is small (e.g., binary digits or nucleobases)

```
Text:     1 0 1 0 1 0 1 0 0
Pattern:  1 0 1 0 0
```

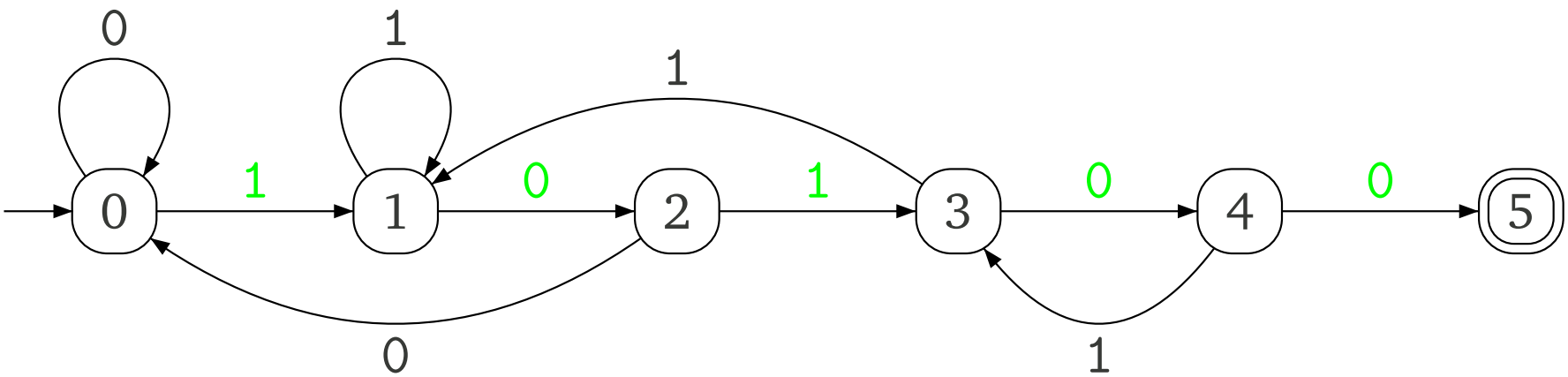
Every “false start” contains a lot of information.

Can we pre-process the pattern to find out when the index i can be incremented by more than 1?

Note: KMP works by comparing from left to right in the pattern (unlike Horspool’s method)

Knuth-Morris-Pratt as Running an FSA

Given the pattern `1 0 1 0 0` we want to construct the following finite-state automaton:

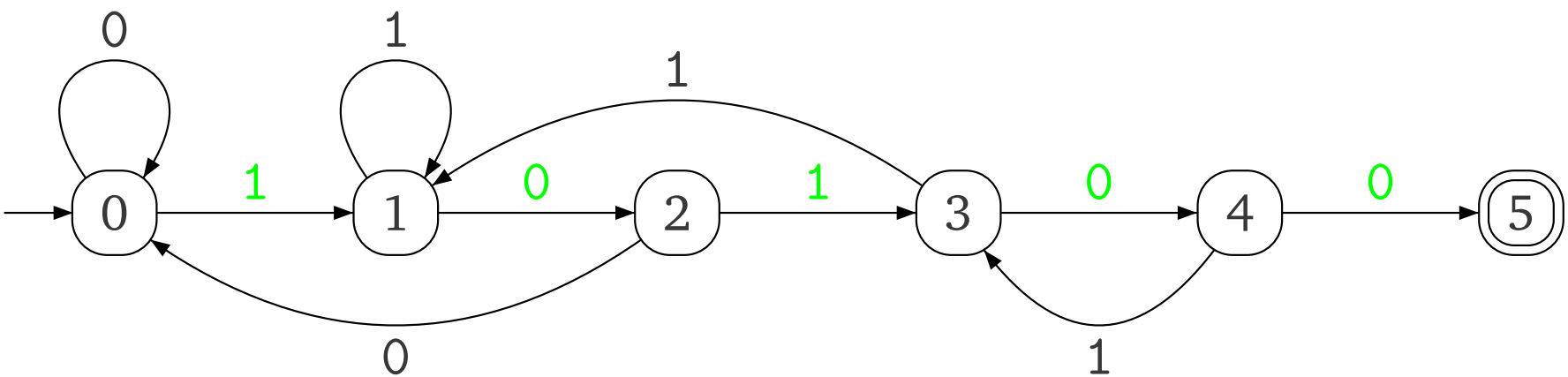


We can capture the behaviour of this automaton in a table.

Knuth-Morris-Pratt Automaton

We can represent the finite-state automaton as a 2-dimensional "transition" array T, where $T[c][j]$ is the state to go to upon reading the character c in state j.

j	T['0'][j]	T['1'][j]
0	0	1
1	2	1
2	0	3
3	4	1
4	5	3



Constructing the Automaton

The automaton (or the table T) can be constructed step-by-step:

Somewhat tricky but fast.

x is a “backtrack point”.

For next state j:

First x’s transitions are copied
(in red).

Then the success arc is updated,
determined by $P[j]$ (in green).

Finally x is updated based on $P[j]$.

Constructing the Automaton

The automaton (or the table T) can be constructed step-by-step:

Somewhat tricky but fast.

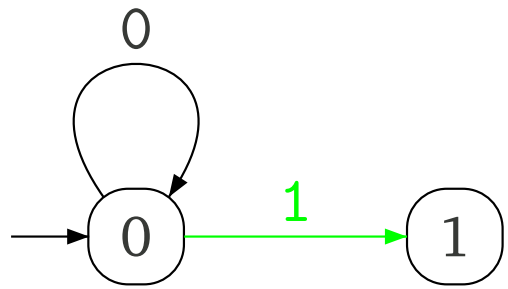
x is a “backtrack point”.

For next state j:

First x’s transitions are copied (in red).

Then the success arc is updated, determined by $P[j]$ (in green).

Finally x is updated based on $P[j]$.



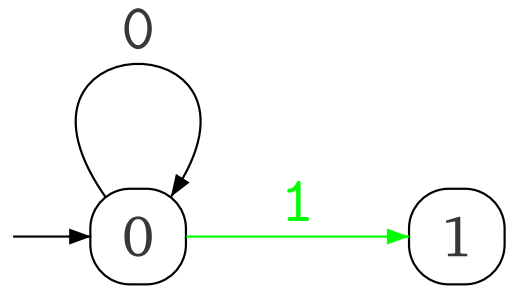
Constructing the Automaton

The automaton (or the table T) can be constructed step-by-step:

Somewhat tricky but fast.

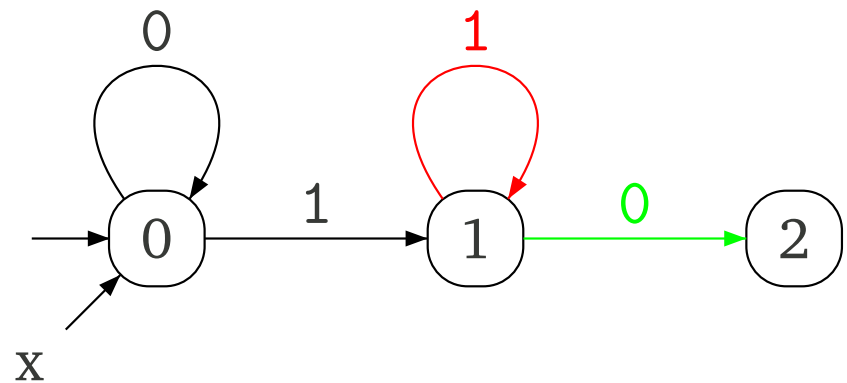
x is a “backtrack point”.

For next state j:



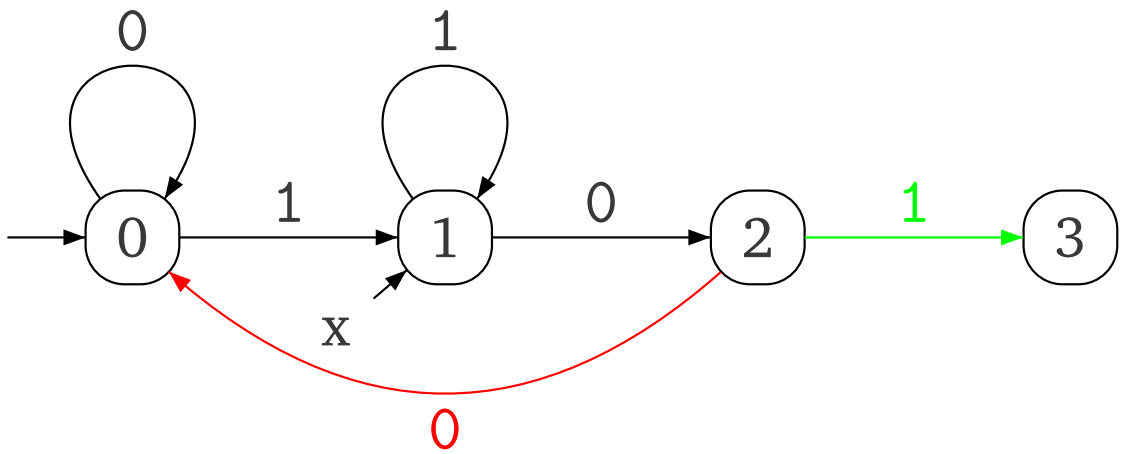
First x’s transitions are copied (in red).

Then the success arc is updated, determined by $P[j]$ (in green).

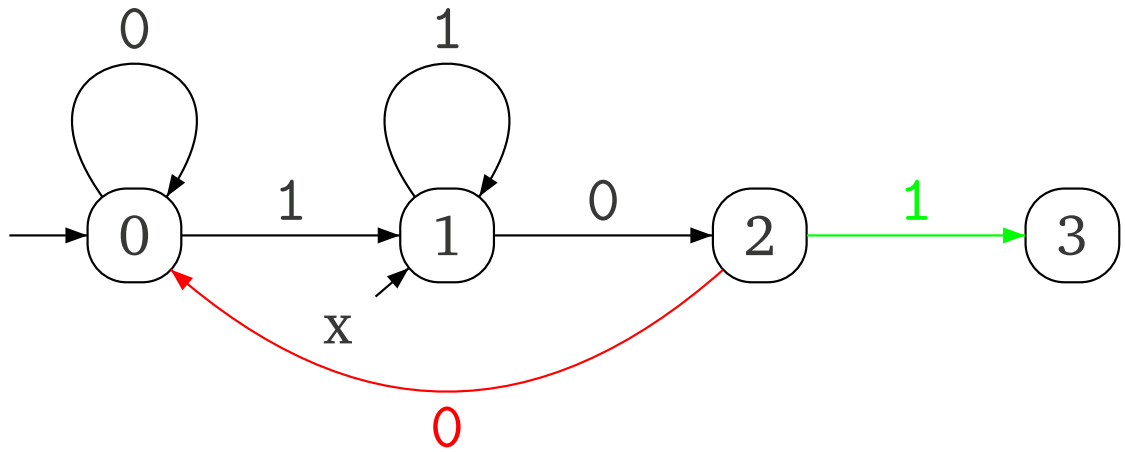


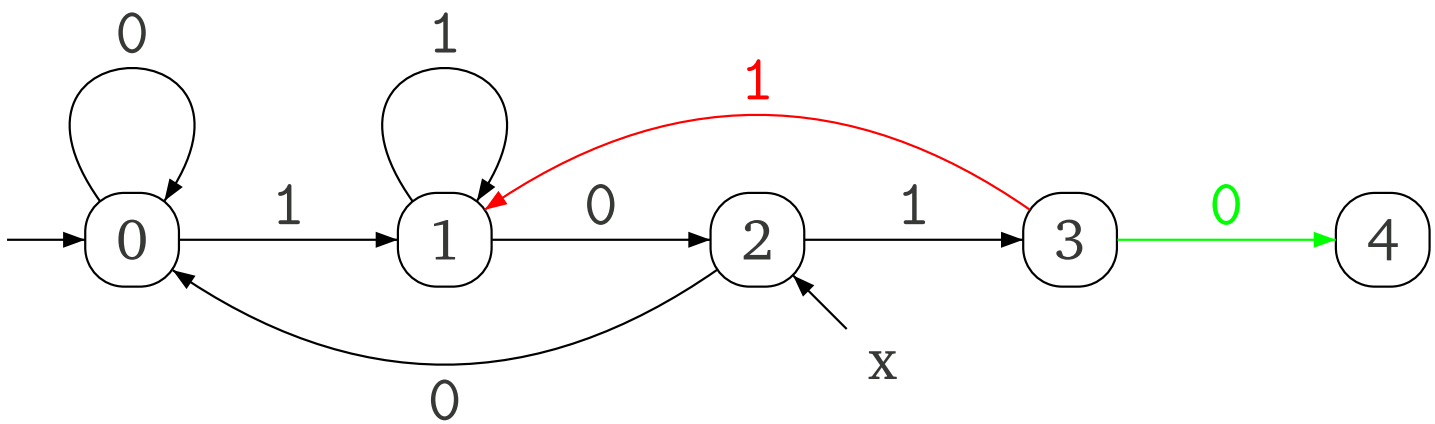
Finally x is updated based on $P[j]$.

Constructing the Automaton

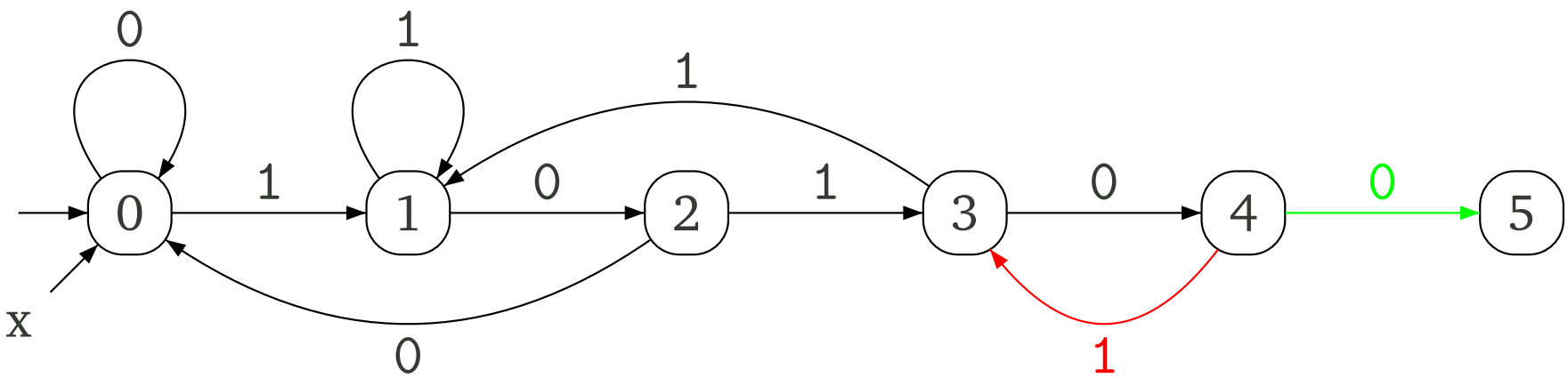


Constructing the Automaton





Constructing the Automaton



Constructing the Automaton

$T['0'][0] \leftarrow 0$
$T['1'][0] \leftarrow 0$
$T[P[0]][0] \leftarrow 1$
$x \leftarrow 0$
$j \leftarrow 1$
while $j < m$ **do**
    $T['0'][j] \leftarrow T['0'][x]$
    $T['1'][j] \leftarrow T['1'][x]$
    $T[P[j]][j] \leftarrow j + 1$
    $x \leftarrow T[P[j]][x]$
    $j \leftarrow j + 1$

Pattern Compilation: Hard-Wiring the Pattern

Even better, directly produce code C code for given pattern.
For the example $p =$ `1 0 1 0 0`:

```
int kmp(char *s) {
    int i = -1;
    s0: i++; if (s[i] == '0') goto s0;
    s1: i++; if (s[i] == '1') goto s1;
    s2: i++; if (s[i] == '0') goto s0;
    s3: i++; if (s[i] == '1') goto s1;
    s4: i++; if (s[i] == '1') goto s3;
    s5: return i-4;
}
```

This assumes that we have posted a sentinel, that is, appended p to the end of s before running `kmp(s)`.

Summary

Today

- Use memory to save time
- Horspool's method
- Overview of other string search

Next

- Hashing